Assignment 5: Ensemble Methods

Copyright and Fair Use

This material, no matter whether in printed or electronic form, may be used for personal and non-commercial educational use only. Any reproduction of this material, no matter whether as a whole or in parts, no matter whether in printed or in electronic form, requires explicit prior acceptance of the authors.

Automatic Testing Guidelines

Automatic unittesting requires you to submit a notebook which contains strictly defined objects. Strictness of definition consists of unified shapes, dtypes, variable names and more.

Within the notebook, we provide detailed instruction which you should follow in order to maximise your final grade.

Name your notebook properly, follow the pattern in the template name:

Assignment_N_NameSurname_matrnumber

1. N - number of assignment
2. NameSurname - your full name where every part of the name starts with a capital letter, no spaces
3. matrnumber - you student number on ID card (with k, potentially with a leading zero)

Don't add any cells but use the ones provided by us. All cells have a unique ID so that the unit test can find it, so please do not add or remove any cell!

Always make sure that implemented functions have the correct output and given variables contain the correct data type. In the descriptions for every function you can find information on what datatype an output should have and you should stick to that in order to minimize conflicts with the unittest. Don't import any other packages than listed in the cell with the "imports" tag.

Questions are usually multiple choice (except the task description says otherwise) and can be answered by changing the given variables to either "True" or "False". "None" is counted as a wrong answer in any case!

Note: Never use variables you defined in another cell in your functions directly; always pass them to the function as a parameter. In the unitest, they won't be available either. If you want to make sure that everything is executable for the unittest, try executing cells/functions individually (instead of running the whole notebook).

Task 1: AdaBoostM1 is an instance of forward stagewise modelling

In the lecture it was mentioned that one of the first boosting algorithms, i.e. AdaBoostM1, is equivalent to forward stagewise modelling using the exponential loss $L(y, g(\mathbf{x})) = \exp(-yg(\mathbf{x}))$ for a binary classification problem ($y \in \{-1, 1\}$). In this task we intend to provide proof of this fact. We will guide you through the most important steps and you will have to add some details.

For AdaBoostM1, the basis functions at timestep n are the individual classifiers $b_n(\mathbf{x}) \in \{-1, 1\}$. We assume that all of them are slightly better than random guessing. Note that we use b_n here for the resulting classifier at timestep n, which differs slightly from the notation in the slides, mainly to not confuse it with the corresponding approximation from forward stagewise modelling, which is also called g_n there.

Using the exponential loss in each timestep n we have to solve $(\beta_n, b_n) = \arg\min_{\beta,b} \sum_{i=1}^{l} \exp(-y_i(g_{n-1}(\mathbf{x}_i) + \beta b(\mathbf{x}_i))$ for the classifier b_n and the coefficient β_n which are added at each step.

This can be rewritten as: $(\beta_n, b_n) = \arg\min_{\beta,b} \sum_{i=1}^{l} w_i^{(n)} \exp(-y_i \beta b(\mathbf{x}_i)), \qquad (1)$

with $w_i^{(n)} = \exp(-y_i g_{n-1}(\mathbf{x}_i))$.

Since each $w_i^{(n)}$ depends neither on β nor b, it can be regarded as a weight that is applied to each observation. This weight depends on $g_{n-1}(\mathbf{x}_i)$, and so the individual weight values change with each iteration n. The solution of (1) can be found in two steps:

Calculation 1.1 (10 Points):

In the **first step** fix $\beta \geq 0$ and show that in this case the solution to (1) is $b_n = \arg\min_b \sum_{i=1}^{l} w_i^{(n)} I(b(\mathbf{x}_i) \neq y_i)$.

Hints: Try to write the expressions in (1) after $\arg\min$ in the form $\sum_{b(\mathbf{x}_i)=y_i}(\dots) + \sum_{b(\mathbf{x}_i)\neq y_i}(\dots)$.

Find the right expressions for (...), such that the product $b(\mathbf{x}_i)y_i$ doesn't appear there anymore.

Now show that this can be written as:
$(\exp(\beta) - \exp(-\beta)) \sum_{b(\mathbf{x}_i)\neq y_i} w_i^{(n)} + \exp(-\beta) \sum_{i=1}^{l} w_i^{(n)}. \qquad (2)$

The equation $\sum_i w_i^{(n)} = \sum_{b(\mathbf{x}_i)\neq y_i} w_i^{(n)} + \sum_{b(\mathbf{x}_i)=y_i} w_i^{(n)}$ might be helpful.

Argue why this already implies the claim, i.e., the solution to (1) is $b_n = \arg\min_b \sum_{i=1}^{l} w_i^{(n)} I(b(\mathbf{x}_i) \neq y_i)$.

Please provide reasoning and explanations in full sentences. Grading of the task will heavily depend on it.

SOLUTION:

for $b(x_i) = y_i, \quad y_i b(x_i) = 1, \quad \exp(-\beta) = \exp(\beta)$

for $b(x_i) \neq y_i, \quad y_i b(x_i) = -1, \quad \exp(-\beta) = \exp(-\beta)$

We can write the sum as:

$\sum_{i=1}^{l} w_i^{(n)} \exp(-y_i \beta b(\mathbf{x}_i)) = \sum_{b(\mathbf{x}_i)=y_i} w_i^{(n)} \exp(-\beta) + \sum_{b(\mathbf{x}_i)\neq y_i} w_i^{(n)} \exp(\beta)$

Since the total sum of weights is constant, we can write:

$\sum_{i=1}^{l} w_i^{(n)} = \sum_{b(\mathbf{x}_i)=y_i} w_i^{(n)} + \sum_{b(\mathbf{x}_i)\neq y_i} w_i^{(n)}$

Denoting the third term as A and the first term as B, we can write the middle term as:

$A = \sum_{b(\mathbf{x}_i)\neq y_i} w_i^{(n)} \exp(\beta) \quad \text{and} \quad B = \sum_{i=1}^{l} w_i^{(n)}$

We get:

$\sum_{b(\mathbf{x_i})=y_i} w_i^{(n)} = A - B$

Sub. back to the original equation:

$\sum_{i=1}^{l} w_i^{(n)} \exp(-y_i \beta b(\mathbf{x}_i)) = \sum_{b(\mathbf{x}_i)=y_i} w_i^{(n)} \exp(-\beta) + \sum_{b(\mathbf{x}_i)\neq y_i} w_i^{(n)} \exp(\beta)$

which is equal to with A and B:

$$(A - B)\exp(-\beta) + A\exp(\beta)$$

Grouping the terms we get:

$$A(\exp(\beta) - \exp(-\beta)) + B\exp(-\beta) \qquad (2)$$

Calculation 1.2 (20 Points):

In the **second step** you need to optimize the following expression with respect to β:

$$(\exp(\beta) - \exp(-\beta)) \sum_{b_n(\mathbf{x}_i)\neq y_i} w_i^{(n)} + \exp(-\beta) \sum_{i=1}^{l} w_i^{(n)}$$

Hint: do it in the usual way (i.e. by differentiating the expression and setting it to 0).

Using the abbreviation $\text{err}_n = \frac{\sum_{i=1}^{l} w_i^{(n)} I(y_i \neq b_n(\mathbf{x}_i))}{\sum_{i=1}^{l} w_i^{(n)}}$ show that the obtained expression is $\beta_n = \frac{1}{2}\ln\frac{1-\text{err}_n}{\text{err}_n}$.

Note that $\beta_n \geq 0$, by our assumption that all classifiers are better than random guessing, i.e. $\text{err}_n \leq \frac{1}{2}$, so the result is in accordance with the previous subtask.

Please provide reasoning and explanations in full sentences. Grading of the task will heavily depend on it.

SOLUTION:

$$(\exp(\beta) - \exp(-\beta)) \sum_{b_n(\mathbf{x}_i) \neq y_i} w_i^{(n)} + \exp(-\beta) \sum_{i=1}^{l} w_i^{(n)}$$

We need to differentiate the expression w.r.t β and set it to 0:

$$\frac{d}{d\beta}((\exp(\beta) - \exp(-\beta))A + \exp(-\beta)B) = 0$$

We get the equation:

$$A(\exp(\beta) + \exp(-\beta)) - B\exp(-\beta) = 0$$

Rearranging the terms we get:

$$A\exp(\beta) + (A - B)\exp(-\beta) = 0$$

Dividing both sides with $\exp(-\beta)$ we get:

$$A\exp(2\beta) = B - A$$

$$\exp(2\beta) = \frac{B - A}{A}$$

We take the nat. log of both sides and get:

$$2\beta = \ln\left(\frac{B - A}{A}\right)$$

We can express the error in terms of A and B:

$$\text{err}_n = \frac{\sum_{i=1}^{l} w_i^{(n)} I(y_i \neq b_n(\mathbf{x}_i))}{\sum_{i=1}^{l} w_i^{(n)}} = \frac{A}{B}$$

$$B\ \text{err}_n = A$$

We can write the expression as:

$$\beta_n = \frac{1}{2}\ln\left(\frac{1 - \text{err}_n}{\text{err}_n}\right)$$

Calculation 1.3 (10 Points):

In the **final step** we can update the approximation as follows:
$g_n(\mathbf{x}) = g_{n-1}(\mathbf{x}) + \beta_n b_n(\mathbf{x})$. To finish the proof proceed by deriving the following

relations:

The weights for the next generation can be computed as follows:

$$w_i^{(n+1)} = w_i^{(n)} \exp(-y_i \beta_n b_n(\mathbf{x}))$$

$$-y_i b_n(\mathbf{x}) = 2I(y_i \neq b_n(\mathbf{x}_i)) - 1$$

Use these two relations to show that:

$$w_i^{(n+1)} = w_i^{(n)} \exp(-\beta_n) \exp(\alpha_n I(y_i \neq b_n(\mathbf{x}_i))) \qquad (2)$$

where $\alpha_n = 2\beta_n$ is the α_n from the AdaBoostM1 algorithm from the lecture.

Please provide reasoning and explanations in full sentences. Grading of the task will heavily depend on it.

SOLUTION:

From the weights of the new generation $-y_i b_n(\mathbf{x}) = 2I(y_i \neq b_n(\mathbf{x}_i)) - 1$:

Plugging in $w_i^{(n+1)}$ we get:

$$w_i^{(n+1)} = w_i^{(n)} \exp(-\beta_n(2I(y_i \neq b_n(\mathbf{x}_i)) - 1))$$

Arranging the terms we get:

$$w_i^{(n+1)} = w_i^{(n)} \exp(\beta_n) \exp(-2\beta_n I(y_i \neq b_n(\mathbf{x}_i)))$$

Denoting $\alpha_n = 2\beta_n$ we get:

$$w_i^{(n+1)} = w_i^{(n)} \exp(-\beta_n) \exp(\alpha_n I(y_i \neq b_n(\mathbf{x}_i))) \qquad (2)$$

Task 2: Random Forests and Feature importance

In this task you will train a Random Forest (RF) Classifier on a subset of fashionMNIST. You should observe how these models can immediately give you useful information about feature importance, which is a very convenient property of RFs.

- **Code 2.1**:
  - Firstly, implement a function called `filter_dataset()` that filters the data set for the classes with labels 3 (dresses) and 8 (bags). You should create the filtered train data set from `X_train` and `y_train` and the test data set from `X_test` and `y_test`.
    Hint: Masks provide a convenient solution to this task.
  - After the filtering procedure the data samples corresponding to dresses should be labelled as 0 and the bags as 1.
- **Code 2.2**:
  - Your task now is to train a sklearn RandomForestClassifier with the default parameters on the training data set.

    - Then get the model's predictions for the test data set. Use the given seed as random_seed for the `RandomForestClassifier`.
    - For this task, we ask you to implement the function `fit_predict()`.
- **Code 2.3**:
    - Now, by implementing the function `get_n_wrong_samples()`, compute and return the size of the test set and number of misclassified test samples.
    - Furthermore, retrieve the misclassifed samples, their actual labels and the predicted labels from the classifier.
    - Use your predictions to plot up to 20 test data sample(s) that were **missclassified** (there can be fewer than 20 misclassified samples as well):
- **Plot 2.4**:
    - Within this part we will try to see the decision-making incentives of Random Forest. To do this we ask you to implement the following tasks in the function `plot_heatmaps()`:
      Hint: In the following you will have to plot heatmaps, to do this you might want to take a look at seaborn (sns): Seaborn heatmap
        1. Take your training dataset and split it into 2 parts: dresses and bags. Calculate the average of the features. Reshape the averages to 2D arrays of shape 28*28 and plot them as heatmaps.
        2. From the average of dresses subtract the average of bags. Plot the result as a heatmap.
        3. Define the feature importance of the previously trained RF classifier. Visualize it as a heatmap and don't forget to reshape it to 28*28.
           Hint: Check scikit-learn documentation to access feature importance.
- **Question 2.5**:
    - Finally, answer some questions about the results of the previous exercises.

Code 2.1 (15 Points):

In [1]:
```
# Nothing to do here, just run the cell.
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from mnist_loader import MNIST
import matplotlib.pyplot as plt
import seaborn as sns
# Set random seed to ensure reproducible runs
RSEED = 10
```

In [2]:
```
# Nothing to do here, just run the cell.
data = MNIST('./dataset/')
img_train, labels_train = data.load_training()
X_train = np.array(img_train)
y_train = np.array(labels_train)
print(X_train.shape, y_train.shape)

X_test, y_test = data.load_testing()
X_test = np.array(X_test)
y_test = np.array(y_test)
print(X_test.shape, y_test.shape)
```

```
(60000, 784) (60000,)
(10000, 784) (10000,)
(10000, 784) (10000,)
```

In [3]:

```python
def filter_dataset(X_train: np.ndarray, y_train: np.ndarray, X_test: np.n
    """This function filters the datasets w.r.t to given labels_list.

    So, in the end X_train and X_test only contain the samples with label

    Parameters
    ----------
    X_train : np.ndarray
        Training data matrix.
    y_train : np.ndarray
        Training labels vector.
    X_test : np.ndarray
        Test data matrix.
    y_test : np.ndarray
        Test labels vector.
    labels_list : list[int, int]
        List of length 2 which consists of integer labels.

    Returns
    -------
    tuple of np.ndarray
        - X_train_filtered : np.ndarray
            The train samples with either label "Dress" or "Bag".
        - y_train_filtered : np.ndarray
            The train label vector containing only 0s and 1s.
        - X_test_filtered : np.ndarray
            The train samples with either label "Dress" or "Bag".
        - y_test_filtered : np.ndarray
            The test label vector containing only 0s and 1s.
    """
    # YOUR CODE HERE
    X_train_filtered = []
    y_train_filtered = []
    X_test_filtered = []
    y_test_filtered = []

    label_map = {labels_list[0]: 0,
                 labels_list[1]: 1}

    for i in range(len(y_train)):
        if y_train[i] in labels_list:
            X_train_filtered.append(X_train[i])
            y_train_filtered.append(label_map[y_train[i]])

    for i in range(len(y_test)):
        if y_test[i] in labels_list:
            X_test_filtered.append(X_test[i])
            y_test_filtered.append(label_map[y_test[i]])

    X_train_filtered = np.array(X_train_filtered)
    y_train_filtered = np.array(y_train_filtered)
    X_test_filtered = np.array(X_test_filtered)
    y_test_filtered = np.array(y_test_filtered)

    return X_train_filtered, y_train_filtered, X_test_filtered, y_test_fi
```

In [4]:
```
# DO NOT DELETE THIS CELL!
X_train_testing = np.arange(10).reshape(10, -1)
y_train_testing = np.concatenate((np.array([3]*2), np.array([8]*2), np.ar
X_test_testing = np.arange(20, 30).reshape(10, -1)
y_test_testing = np.concatenate((np.array([2]*1), np.array([3]*4), np.arr

X_train_filtered, y_train_filtered, X_test_filtered, y_test_filtered = fi

assert isinstance(X_train_filtered, np.ndarray), "The filtered X_train ma
np.testing.assert_array_equal(X_train_filtered, np.array([[0], [1], [2],
```

In [5]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(y_train_filtered, np.ndarray), "The filtered y_train ve
np.testing.assert_array_equal(y_train_filtered, np.array([0, 0, 1, 1]))
```

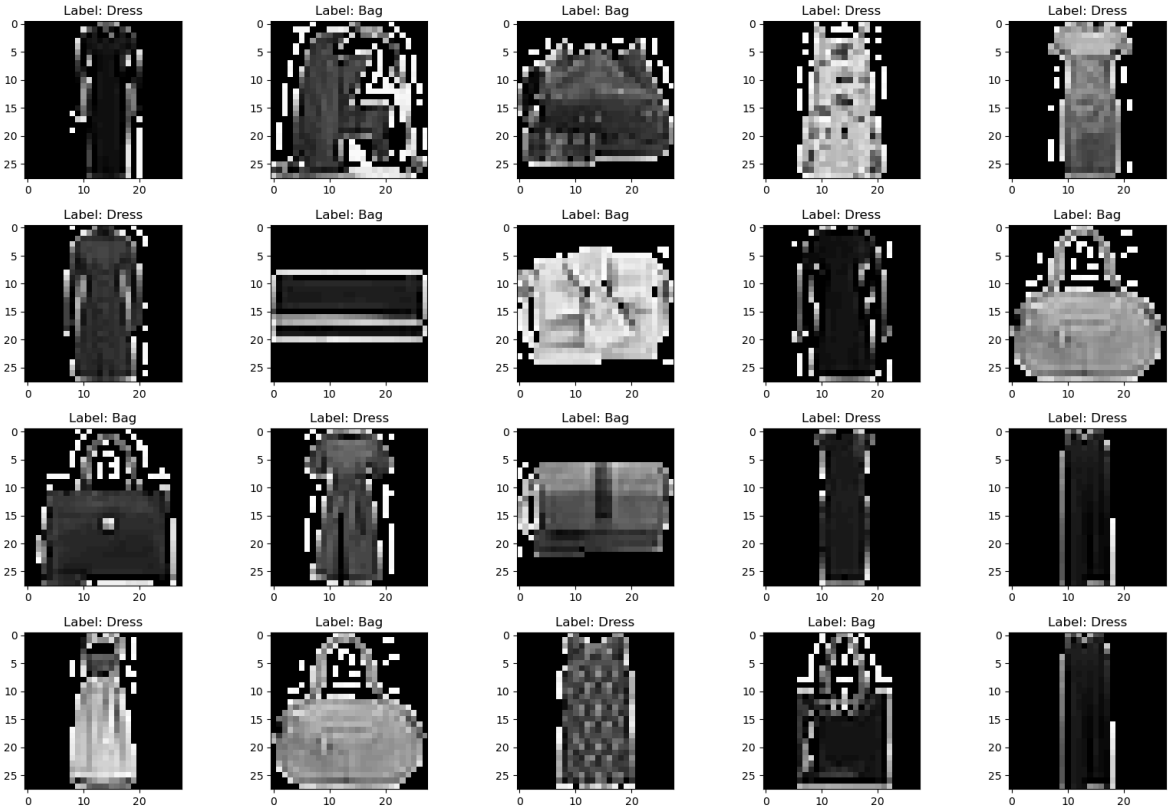
In [6]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(X_test_filtered, np.ndarray), "The filtered X_test matr
np.testing.assert_array_equal(X_test_filtered, np.array([[21], [22], [23]
```

In [7]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(y_test_filtered, np.ndarray), "The filtered y_test vect
np.testing.assert_array_equal(y_test_filtered, np.array([0, 0, 0, 0, 1]))
```

In [8]:
```
# Nothing to do here, just run the cell.
X_train_filtered, y_train_filtered, X_test_filtered, y_test_filtered = fi
```

In the following we visualize a few randomly selected samples from our filtered training data:

In [9]:
```
# Nothing to do here, just run the cell.
# A routine that you can use for plotting some of the data.
arr = ['Dress', 'Bag'] # 0: Dress,  1: Bag
a = np.random.randint(1,40,20)
plt.figure(figsize=(20, 13))
for n,i in enumerate(a):
    plt.subplot(4, 5, n+1)
    two_d = (np.reshape(X_train_filtered[i], (28, 28)) * 255).astype(np.u
    plt.title('Label: {0}'.format(arr[y_train_filtered[i]]))
    plt.imshow(two_d, interpolation='nearest',cmap='gray')
plt.subplots_adjust(hspace = 0.3)
```



Code 2.2 (5 Points):

In [10]:
```python
def fit_predict(X_train: np.ndarray, y_train: np.ndarray, X_test: np.ndar
    """Function fits a RandomForestClassifier on the training data and re

    Parameters
    ----------
    X_train : np.ndarray
        Training data matrix.
    y_train : np.ndarray
        Training labels vector.
    X_test : np.ndarray
        Test data matrix.
    seed : int
        Random seed used for initializing the classifier.

    Returns
    -------
    tuple of RandomForestClassifier, np.ndarray
        - model : RandomForestClassifier
            The fitted classifier.
        - prediction : np.ndarray
            The prediction on the given test set.
    """
    # YOUR CODE HERE
    model = RandomForestClassifier(random_state=seed)
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)

    return model, prediction
```

In [11]:
```python
# DO NOT DELETE THIS CELL!
model, prediction = fit_predict(X_train[:1000], y_train[:1000], X_test[:1
```

```
assert isinstance(model, RandomForestClassifier), "The fitted model is no
assert isinstance(prediction, np.ndarray), "The prediction is not a np.nd
assert len(prediction) == 1000, "The number of predictions is wrong!"
np.testing.assert_array_equal(prediction[:5], np.array([0, 1, 2, 2, 1]))
np.testing.assert_array_equal(prediction[-5:], np.array([9, 2, 9, 4, 5]))
```

In [12]:
```
# Nothing to do here, just run the cell.
model, prediction = fit_predict(X_train_filtered, y_train_filtered, X_tes
```

Code 2.3 (15 Points):

In [13]:
```
def get_n_wrong_samples(X_test: np.ndarray, y_test: np.ndarray, predictio
    """Calculates the size of the test set, number of misclassified sampl
    the actual labels of the misclassified samples, and the predicted lab

    Parameters
    ----------
    X_test : np.ndarray
        Test data matrix.
    y_test : np.ndarray
        Test labels vector.
    prediction : np.ndarray
        The predicted labels vector from the model.

    Returns
    -------
    tuple of int, np.ndarray
        - test_size : int
            The size of the test set.
        - num_wrong : int
            The number of misclassified samples.
        - wrong_samples : np.ndarray
            The misclassified samples.
        - wrong_labels : np.ndarray
            The actual labels of the misclassified samples.
        - wrong_predictions : np.ndarray
            The predicted incorrect labels.
    """
    # YOUR CODE HERE
    test_size = len(y_test)
    num_wrong = 0
    wrong_samples = []
    wrong_labels = []
    wrong_predictions = []

    for i in range(test_size):
        if y_test[i] != prediction[i]:
            num_wrong += 1
            wrong_samples.append(X_test[i])
            wrong_labels.append(y_test[i])
            wrong_predictions.append(prediction[i])

    wrong_samples = np.array(wrong_samples)
    wrong_labels = np.array(wrong_labels)
    wrong_predictions = np.array(wrong_predictions)

    return test_size, num_wrong, wrong_samples, wrong_labels, wrong_predi
```

In [14]:
```
# DO NOT DELETE THIS CELL!
test_size, num_wrong, wrong_samples, wrong_labels, wrong_predictions = ge
assert isinstance(test_size, int), "The size of the test set is not an in
assert test_size == 10, "The size of the test set is incorrect!"
```

In [15]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(num_wrong, int), "The number of misclassified samples i
assert num_wrong == 3, "The number of misclassified samples is incorrect!
```

In [16]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(wrong_samples, np.ndarray), "The misclassified samples
np.testing.assert_array_equal(wrong_samples, np.array([[21], [26], [29]])
```

In [17]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(wrong_labels, np.ndarray), "The misclassified labels ar
np.testing.assert_array_equal(wrong_labels, np.array([3, 0, 0]))
```

In [18]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(wrong_predictions, np.ndarray), "The misclassified pred
np.testing.assert_array_equal(wrong_predictions, np.array([1, 3, 4]))
```

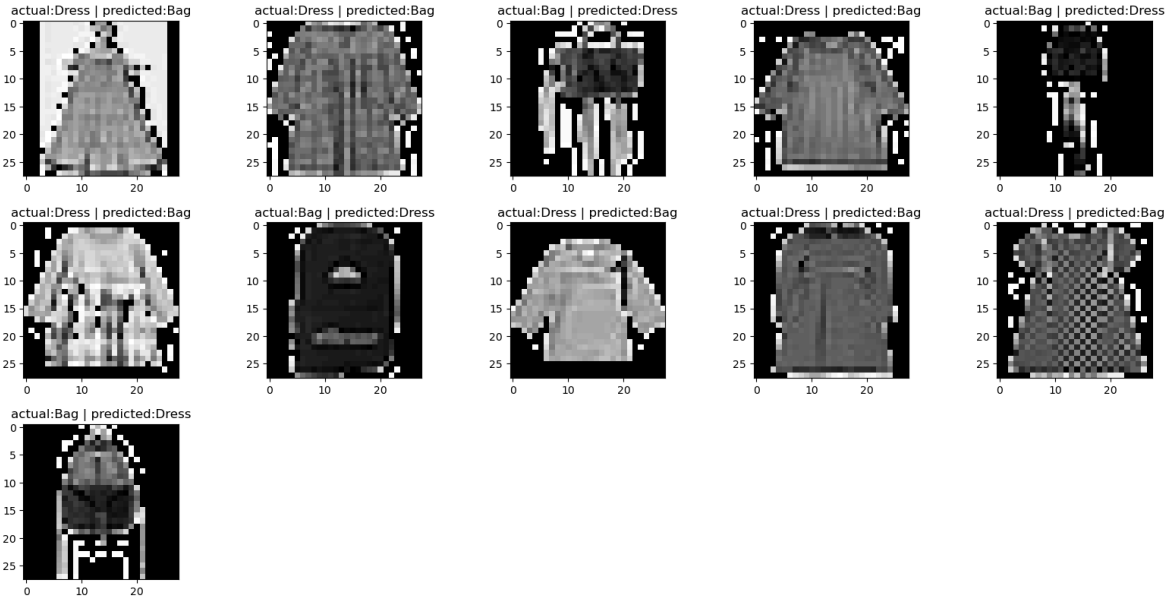
In [19]:
```
# Nothing to do here, just run the cell.
size_test, num_wrong, wrong_samples, wrong_labels, wrong_predictions = ge
print("Number of test samples: {0}\nNumber of misclassified samples: {1}"
```

```
Number of test samples: 2000
Number of misclassified samples: 11
```

In [ ]:
```
def plot_wrong_predictions(wrong_samples: np.ndarray, wrong_labels: np.nd
    """Function plots a figure that shows misclassified samples with thei

    Parameters
    ----------
    wrong_samples : np.ndarray
        The samples that got misclassified by the classifier.
    wrong_labels : np.ndarray
        The actual true labels of the misclassified samples.
    wrong_predictions : np.ndarray
        The predicted labels of the misclassified samples.
    """
    # YOUR CODE HERE
    samples = ['Dress', 'Bag']
    plt.figure(figsize=(20, 13))
    for n,i in enumerate(range(len(wrong_samples))):
        plt.subplot(4,5,n+1)
        matrix = (np.reshape(wrong_samples[i], (28, 28)) * 255).astype(np
        plt.title('actual:{0} | predicted:{1}'.format(samples[wrong_label
                                                      samples[wrong_pre
        plt.imshow(matrix, cmap='gray', interpolation='nearest')
    plt.subplots_adjust(hspace = 0.3)
```

In [21]:
```
# Nothing to do here, just run the cell.
plot_wrong_predictions(wrong_samples, wrong_labels, wrong_predictions)
```



Plot 2.4 (20 Points):

In [22]:
```
def plot_heatmaps(X_train: np.ndarray, y_train: np.ndarray, model: Random
    """Create a plot as described in the task description above.

    Parameters
    ----------
    X_train : np.ndarray
        Training data matrix.
    y_train : np.ndarray
        Training labels vector.
    model : RandomForestClassifier
        Your already trained classifier.
    """
    # YOUR CODE HERE
    dress = X_train[y_train == 0]
    bag = X_train[y_train == 1]

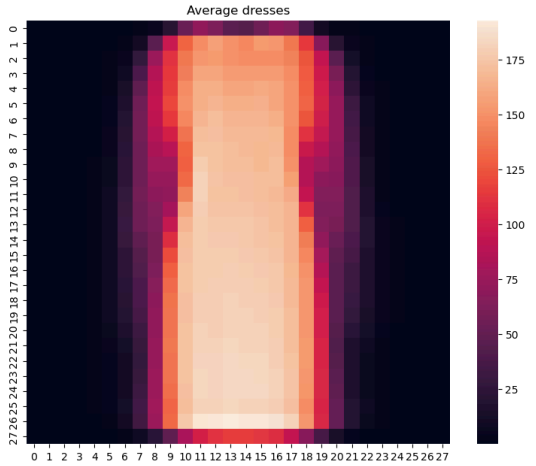
    dress_mean = np.mean(dress, axis=0).reshape(28, 28)
    bag_mean = np.mean(bag, axis=0).reshape(28, 28)

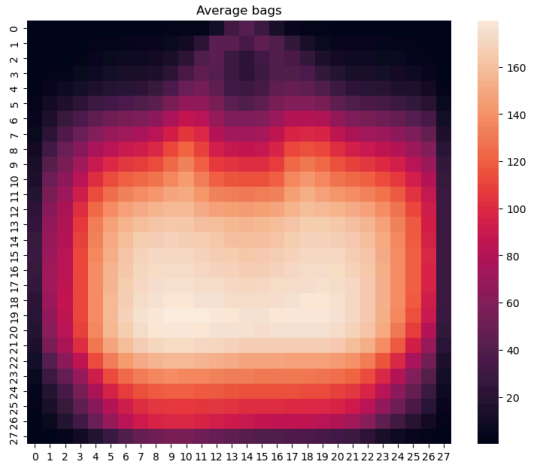
    diff = dress_mean - bag_mean
    feat_imp = model.feature_importances_.reshape(28, 28)

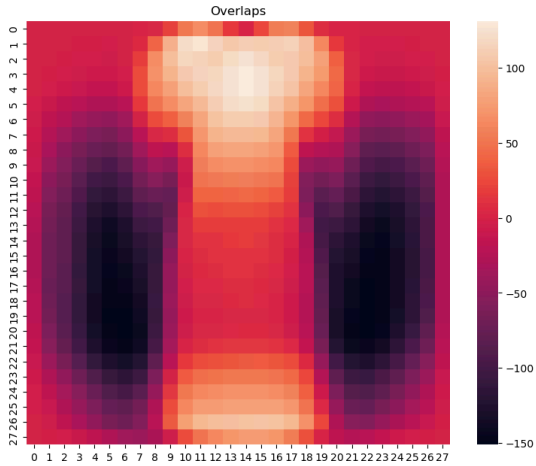
    fig, ax = plt.subplots(2, 2, figsize=(20, 20))

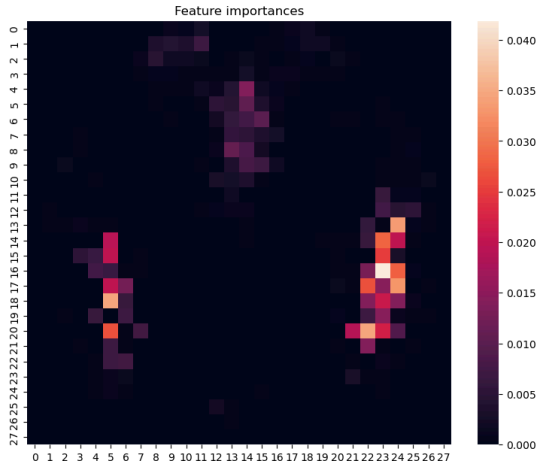
    sns.heatmap(dress_mean, ax=ax[0, 0], square=True, cbar=True, cbar_kws
    ax[0, 0].set_title('Average dresses')
    sns.heatmap(bag_mean, ax=ax[0, 1], square=True, cbar=True, cbar_kws={
    ax[0, 1].set_title('Average bags')
    sns.heatmap(diff, ax=ax[1, 0], square=True, cbar=True, cbar_kws={'shr
    ax[1, 0].set_title('Overlaps')
    sns.heatmap(feat_imp, ax=ax[1, 1], square=True, cbar=True, cbar_kws={
    ax[1, 1].set_title('Feature importances')
    plt.show()
```

In [23]:
```
# Nothing to do here, just run the cell.
plot_heatmaps(X_train_filtered, y_train_filtered, model)
```

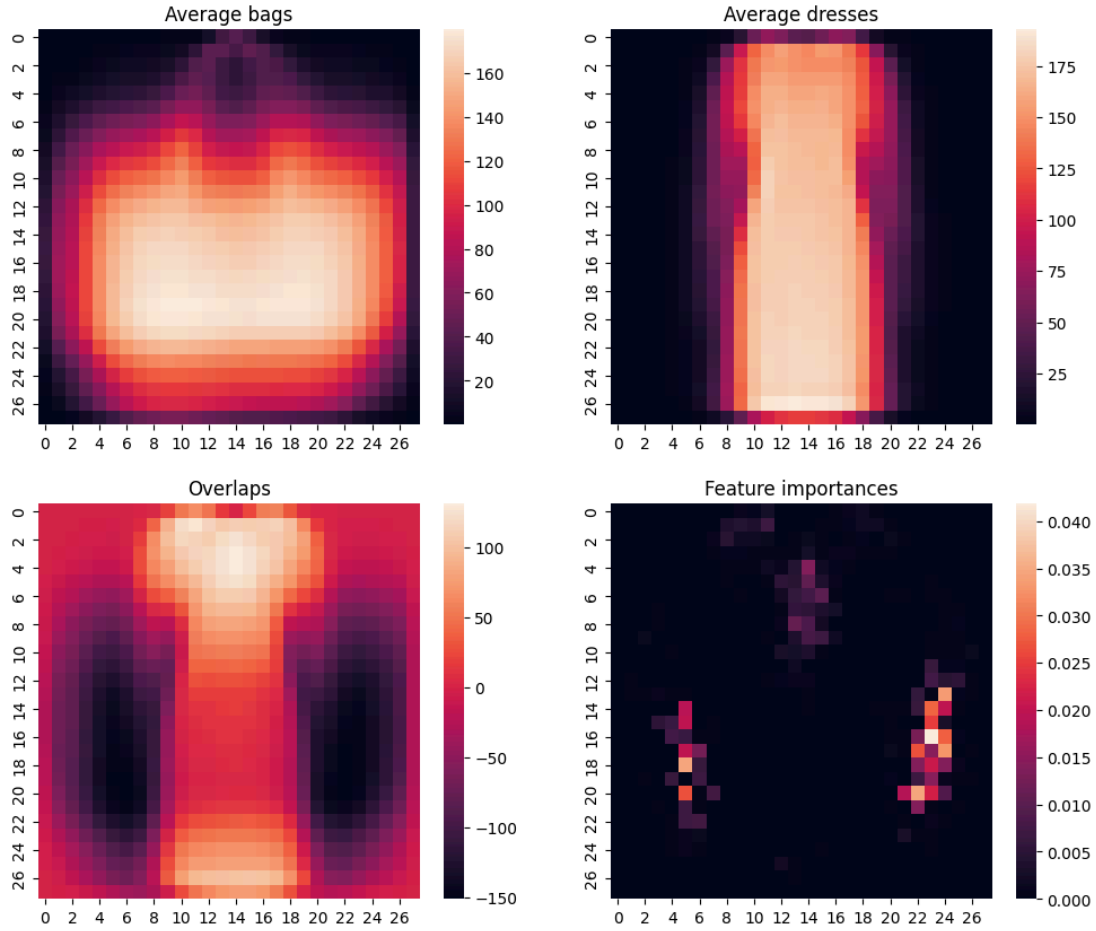








If you have solved the previous task correctly, the resulting plot should look close to this:



Question 2.5 (5 Points):

What observations can you make?
(Multiple answers might be correct)

a_) RF achieves an accuracy higher than 90% on the test set.
b_) The most important features are located in image regions that primarily show either only bags or dresses as depicted by the plots of average bags and dresses.
c_) If one pixel would always be bright for bags but never for dresses, the RF classifier would certainly learn to use only this pixel as a feature.
d_) Comparing the misclassified bags to other bags suggests that they might be outliers as they are rather tall, but not as long as most other bags.

To answer the question, assign `True` or `False` boolean values to variables in the next cell. For example, if you think that **a_)** is correct, define a variable `a_` and set it to `True`, the same applies to **b_)** and the other options. A non-correctly answered question as well as no answer (i.e. answer "None") yields 0 points for a specific question.

In [24]:
```
# YOUR CODE HERE
a_ = True
b_ = False
```

```
c_ = True
d_ = False
```

In [25]:
```
# DO NOT DELETE THIS CELL!
assert a_ is not None, "Store True/False!"
assert a_ in [True, False], "Invalid Answer!"
```

In [26]:
```
# DO NOT DELETE THIS CELL!
assert b_ is not None, "Store True/False!"
assert b_ in [True, False], "Invalid Answer!"
```

In [27]:
```
# DO NOT DELETE THIS CELL!
assert c_ is not None, "Store True/False!"
assert c_ in [True, False], "Invalid Answer!"
```

In [28]:
```
# DO NOT DELETE THIS CELL!
assert d_ is not None, "Store True/False!"
assert d_ in [True, False], "Invalid Answer!"
```